

Manual Completo de Lógica de Programação

Nome: Anna Beatriz Rodrigues Pinhas
Turma: tdsf1B

SENAI

Introdução

O que é lógica de programação?

Lógica de programação é basicamente a forma de organizar pensamentos e criar uma sequência de passos para resolver determinado problema usando o programação. Ela é a base para desenvolver soluções de maneira lógica e organizada. Envolve conceitos como variáveis, condições, repetições e algoritmos, que permitem o computador executar tarefas corretamente.

Para que ela serve?

A lógica de programação serve para criar soluções organizadas e eficientes para problemas usando o computador. Ela ajuda as pessoas a pensar de forma lógica, definir passos corretos e desenvolver programas que executem suas tarefas corretamente. Além disso, facilita o aprendizado de linguagens de programação e melhora a capacidade de raciocínio e resolução de problemas.

Onde ela é utilizada?

A lógica de programação é utilizada em áreas como inteligência artificial, robótica, automação, bancos de dados e até em equipamentos eletrônicos que precisam seguir instruções para funcionar corretamente. Além da tecnologia, a lógica de programação pode ser usada para organizar tarefas e resolver problemas do dia a dia de forma mais eficiente.

Importância da lógica para programação?

A lógica de programação é importante para os dias atuais pois ajuda o programador a criar soluções organizadas, eficientes e corretas para os problemas. Ela consegue desenvolver o raciocínio lógico, facilita a compreensão das linguagens de programação e reduz os erros durante o desenvolvimento de sistemas. Além disso, é a base para criar programas, aplicativos, sites e outras tecnologias de forma estruturada e eficiente.

Linguagem Natural

O que é?

linguagem natural é a forma de descrever instruções e soluções usando o idioma que as pessoas falam no dia a dia, como português ou inglês. Ela é utilizada para explicar algoritmos e processos de maneira simples e fácil de entender, antes de transformá-los em código de programação.

Exemplo de linguagem comum:

1 Exemplo:

Descobrir se uma pessoa pode votar:

- 1 - Receber a idade da pessoa.
- 2 - Verificar se a idade é maior ou igual a 16.
- 3 - Se for, informar que ela pode votar.
- 4 - Caso contrário, informar que ela não pode votar.

2 Exemplo:

Fazer café:

- 1 - Colocar água na chaleira.
- 2 - Aquecer a água.
- 3 - Colocar o pó de café no filtro.
- 4 - Despejar a água quente no filtro.
- 5 - Servir o café.

Fluxograma

O que é fluxograma?

Fluxograma é uma representação visual de um processo ou algoritmo, feita com símbolos e setas que tem vários significados e que mostram a sequência de passos e decisões. Na lógica de programação, ele ajuda a organizar e entender melhor um problema antes de transformá-lo em código, facilitando o planejamento e a identificação de erros. Ele é muito usado para descrever tarefas simples e complexas, como cálculos, sistemas e processos do dia a dia, tornando mais fácil visualizar como cada etapa acontece e em que ordem.

Para que ele serve?

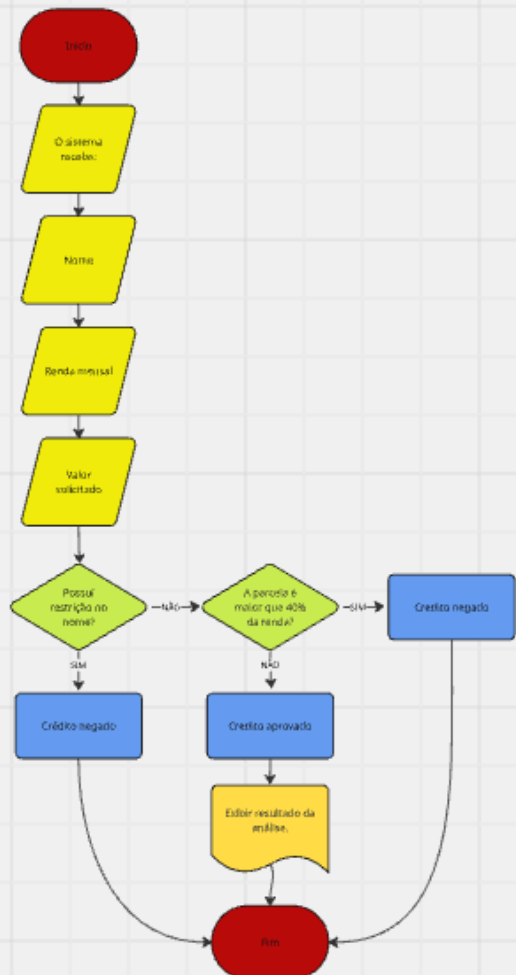
O fluxograma serve para mostrar de forma visual e organizada algum tipo de processo ou algoritmo, facilitando o entendimento de cada etapa e da sequência de ações. Ele serve para planejar soluções antes da programação, identificar erros, melhorar a organização das ideias e comunicar processos de maneira clara entre pessoas que estão desenvolvendo um sistema ou resolvendo um problema.

Principais símbolos utilizados

PRINCIPAIS SÍMBOLOS DO FLUXOGRAMA

SÍMBOLO	NOME	SIGNIFICADO
	TERMINADOR	Indica o início ou o fim do processo.
	PROCESSO	Representa uma ação ou uma atividade que será executada.
	DECISÃO	Indica um ponto de decisão onde há uma pergunta com respostas “sim” ou “não”.
	ENTRADA/SAÍDA	Representa a entrada ou saída de dados do processo.
	LINHA DE FLUXO	Indica a direção e o sentido do fluxo do processo.
	CONECTOR (CONECTOR DE PÁGINA)	Utilizado para conectar partes do fluxograma em outra parte da página.
	CONECTOR (CONECTOR FORA DE PÁGINA)	Utilizado para conectar partes do fluxograma em outras páginas.
	DOCUMENTO	Representa um documento gerado ou utilizado no processo.
	ARMAZENAMENTO DE DADOS	Indica o armazenamento de dados em um banco de dados ou arquivo.

Meus Fluxogramas:



Variáveis

O que são?

Variáveis são espaços usados para guardar informações que podem mudar ao longo do tempo. na programação ela serve para armazenar dados, como números, nomes e textos, ajudando no funcionamento e na organização dos programas.

Tipos de dados:

Na programação, os tipos de dados determinam o que uma variável pode armazenar, por exemplo:]

Inteiro: Armazena números positivos ou negativos sem casas decimais

Real : Armazena números com casas decimais

Cadeia : Armazena letras, palavras ou frases, geralmente delimitadas por aspas

Lógico: Aceita apenas dois valores: Verdadeiro (True) ou Falso (False).

Caractere : Armazena uma única letra, número ou símbolo.

Exemplos:

1 Exemplo:

```
let nome = "Anna"  
let sobrenome = "Beatriz"  
console.log(nome + " " + sobrenome)
```

A variável nesse caso é o nome e o sobrenome, pois eu coloquei let e depois crie a variável colocando nome e sobrenome.

2 Exemplo:

```
let numero1 = 10  
let numero2 = 5  
console.log(numero1 + numero2)
```

Aqui a variável também é o numero1 e o numero2 pela mesma razão.

Entrada e saída de dados

Oque é prompt?

prompt é uma função para exibir uma caixa de diálogo com uma mensagem para você colocar para que assim nós conseguirmos dar um comando ou pergunta digitada para um sistema de inteligência artificial realizar uma tarefa.

Exemplo:

```
let idade = prompt("Qual é a sua idade?",  
"25");
```

```
if (idade >= 18) {  
  console.log("Acesso permitido.");  
} else {  
  console.log("Acesso negado.");  
}
```

o que é console.log?

console.log é um comando do java script que serve para exibir uma mensagem, depois que declarar a variável, usamos console.log para exibir no resultado o que declaramos na variável.

Exemplo:

```
let cidade = "São Paulo"  
let cidade2 = "Rio de Janeiro"
```

```
console.log(cidade2)
```

Operadores

Operadores matemáticos:

Operadores matemáticos são símbolos usados para fazer contas e cálculos em programação e matemática.

Exemplos: =, +, - e etc...

Operadores relacionais:

Operadores relacionais são símbolos usados para comparar valores. O resultado desses símbolos serão sempre verdadeiro ou falso.

Exemplos: ==, !=, >, < e etc...

Operadores lógicos:

Operadores lógicos são usados para juntar ou negar condições em programação. Eles trabalham com valores verdadeiros (true) e falsos (false).

Exemplos:

&&, ||, ! e etc...

Estruturas condicionais

If:

O if é usado para executar um bloco de código somente se uma condição for verdadeira.

Exemplo:

```
// Variável com a idade  
let idade = 18
```

```
// Verifica se a idade é maior ou igual a 18  
if (idade >= 18) {  
  console.log("Pode dirigir")  
}
```

else:

O else é usado quando a condição do if for falsa.

```
// Variável com a idade  
let idade = 16  
  
// Verifica a idade  
if (idade >= 18) {  
  console.log("Pode dirigir")  
} else {  
  console.log("Não pode dirigir")  
}
```

Else if:

O else if serve para adicionar outra condição.
dentro do código.

Exemplo:

```
// Variável com a nota  
let nota = 7
```

```
// Verifica a nota  
if (nota >= 9) {  
  console.log("Excelente")  
} else if (nota >= 6) {  
  console.log("Aprovado")  
} else {  
  console.log("Reprovado")  
}
```

Switch Case

o que é?

switch case é uma estrutura usada para verificar vários casos diferentes de uma variável.

Para que ele
serve?

O switch case serve para verificar vários valores possíveis de uma variável de forma mais organizada e fácil de ler.

Diferença entre
if
e
switch

A diferença entre if e switch é que o if é usado para verificar condições e comparações, como maior, menor ou igual. Ele funciona melhor quando usamos operadores relacionais e lógicos, como >, <, >=, && e ||. Já o switch é usado para comparar valores específicos de uma variável. Ele é mais organizado quando existem várias opções fixas, como dias da semana, cores ou opções de menu.

Exemplo:

```
let opcao = 2
```

```
switch(opcao){  
  case 1:  
    console.log("Perfil")  
    break
```

```
  case 2:  
    console.log("Configurações")  
    break
```

```
  case 3:  
    console.log("Sair")  
    break
```

```
  default:  
    console.log("Opção inválida")  
  }
```

Funções

O que são funções?

Funções são blocos de código criados para executar uma tarefa específica.

Elas ajudam a organizar o código e evitar repetições.

Para que servem?

As funções servem para reutilizar códigos sem precisar escrever tudo novamente várias vezes. Elas também ajudam a organizar melhor o programa, facilitam a manutenção do código e permitem executar tarefas específicas sempre que necessário.

Como criar uma função:

```
function saudacao() {  
  console.log("Olá!")  
}  
saudacao()
```

O que são parâmetros?

Parâmetros são valores que a função recebe para usar dentro dela.

O que é retorno?

O return serve para devolver um valor da função.

```
// Função que soma dois números  
function somar(numero1, numero2) {  
    return numero1 + numero2  
}
```

```
// Guardando o resultado  
let resultado = somar(5, 3)
```

```
console.log(resultado)
```

Exemplos:

```
// Função para verificar idade  
function verificarIdade(idade) {
```

```
    // Verifica se a pessoa é maior de idade  
    if (idade >= 18) {  
        return "Maior de idade"  
    } else {  
        return "Menor de idade"  
    }  
}
```

```
// Chamando a função  
console.log(verificarIdade(20))
```

Comentários no Código

O que são comentários?

Comentários são partes do código que servem para adicionar explicações e observações. Eles não são executados pelo programa.

Para que eles servem?

Os comentários servem para explicar o funcionamento do código, facilitar o entendimento e ajudar outros programadores — ou até o próprio desenvolvedor — a lembrar o que foi feito.

Importância da organização e documentação do código?

Organizar e documentar o código é importante porque deixa o programa mais fácil de entender, corrigir e atualizar. Um código bem organizado facilita a manutenção e o trabalho em equipe.

Comentário de uma linha:

```
// Variável que guarda o nome  
let nome = "Anna"
```

```
console.log(nome)
```

Comentário de várias linhas:

```
/*
```

```
    Esse código verifica  
    se o usuário é maior de idade
```

```
*/
```

```
let idade = 18
```

```
    if (idade >= 18) {  
        console.log("Maior de idade")  
    }
```

Identação de Código

O que é identação?

Identação é a organização do código usando espaços e recuos no início das linhas. Ela serve para mostrar a estrutura do programa e separar blocos de código.

Para que ela serve?

A identação serve para deixar o código mais organizado, facilitando a leitura e o entendimento do programa.

Importância da organização do código

Um código organizado é mais fácil de corrigir, atualizar e entender. Isso ajuda tanto o programador quanto outras pessoas que forem ler o código no futuro.

Como a indentação melhora a leitura e compreensão dos programas?

Com a indentação correta, fica mais fácil identificar:

- onde começam e terminam os blocos;
- quais comandos pertencem a cada condição;
- a estrutura do programa.

Isso evita confusão e ajuda a encontrar erros mais rapidamente.

Exemplo de código organizado:

```
let idade = 18

if (idade >= 18) {
  console.log("Maior de idade")
} else {
  console.log("Menor de idade")
}
```

Exemplo de código desorganizado:

```
let idade=18
if(idade>=18){
console.log("Maior de idade")
}else{
console.log("Menor de
idade")}
```